

mPerm

Setup

```
In[1]:= << mGRG`mPerm`
```

내부용 함수를 간단하게 호출할 수 있도록 축약어 “PP”를 정의한다:

```
In[2]:= $PreRead = ReplaceAll[#,  
    expr_String => StringReplace[expr, "PP" -> "mGRG`mPerm`Private"]] &;
```

Basic Rutines

텐서 인덱스를 교환할 때 대칭인 경우와 반대칭인 경우를 구분해야 한다. 그런데 Mathematica 버전 8에서 도입되었고, 버전 9에서 수정된 ‘순열’ 관련 함수에는 그러한 구분이 없다. (Mathematica 가이드의 Permutations 항목 참고.) 따라서 mPerm 패키지는 Mathematica에 구현된 ‘부호없는 순열’ 처리 함수들을 이용하여 ‘부호있는 순열’을 다룰 수 있도록 확장한다.

함수 이름에 대한 규약: 대문자로 시작하는 함수는 Mathematica 세션에서 전역적으로 사용 가능한 함수이고, 소문자로 시작하면 패키지 내부에서 (개발자가) 사용하도록 설정된 함수이다. (내부용으로 사용되도록 설정된 함수도 “패키지이름`Private`소문자함수” 형태로 호출할 수는 있다.)

Permutation in Mathematica

The result is {4,3,1,2,7,6,5} in MMA Ver. 9. But, {3,4,2,1,7,6,5} in **Ver. 8:**

```
In[3]:= PermutationList[Cycles[{{1, 4, 2, 3}}, {5, 7}]]
```

```
Out[3]= {4, 3, 1, 2, 7, 6, 5}
```

```
In[4]:= % // Ordering
```

```
Out[4]= {3, 4, 2, 1, 7, 6, 5}
```

```
In[5]:= % // Ordering
```

```
Out[5]= {4, 3, 1, 2, 7, 6, 5}
```

permCyclesQ (PermutationCyclesQ) and gensetQ

```
In[6]:= PermutationCyclesQ[Cycles[{{1, 3}}, {2, 5}]]
```

```
Out[6]= True
```

A ‘CyclesPhased’ is defined by a list {Cycles[{{...}}, s], where $|s| = 1$.

```
In[7]:= PP`permCyclesQ[{Cycles[{}], ei Re[θ]}] (* is a CyclesPhased? *)
```

```
Out[7]= True
```

```
In[8]:= PP`permCyclesQ[{Cycles[{{1, 3}, {2, 5}}], ei Re[θ]}]
```

```
Out[8]= True
```

```
In[9]:= PP`permCyclesQ[{Cycles[{{1, 3}, {2, 5}}], 2}] (* |s| ≠ 1 *)
```

```
Out[9]= False
```

```
In[10]:= PP`permCyclesQ[{Cycles[{{1, 3}, {2, 5, 0}}], ei Re[θ]}]
```

... Cycles: Cycles [{{1, 3}, {2, 5, 0}}] contains integers that are not positive.

```
Out[10]=
```

```
False
```

```
In[11]:= PP`permCyclesQ[{Cycles[{{1, 3}, {1, 5}}], ei Re[θ]}]
```

... Cycles: Cycles [{{1, 3}, {1, 5}}] contains repeated integers.

```
Out[11]=
```

```
False
```

```
In[12]:= PP`permCyclesQ[{Cycles[{{1, 3, 1}, {2, 5}}], ei Re[θ]}]
```

... Cycles: Cycles [{{1, 3, 1}, {2, 5}}] contains repeated integers.

```
Out[12]=
```

```
False
```

```
In[13]:= GenSet::usage
```

```
Out[13]=
```

```
GenSet[p1, p2, ...] represents a set of generators for a permutation group.
```

```
In[14]:= PP`gensetQ[GenSet[{Cycles[{{1, 3}, {2, 5}}], ei Re[θ]}]
```

```
Out[14]=
```

```
True
```

PermMax (PermutationMax)

```
In[15]:= PermutationMax[Cycles[{{1, 2}, {4, 5}}]]
```

```
Out[15]=
```

```
5
```

In[16]:= **PermMax::usage**

Out[16]=

PermMax[perm] gives the largest element moved by the permutation 'perm', which can be in CyclesPhased, Imag, or GenSet form.

In[17]:= PermMax[{Cycles[{{1, 2}, {4, 5}}], $e^{i \operatorname{Re}[\theta]}$ }]

Out[17]=

5

In[18]:= PermMax[$e^{i \operatorname{Re}[\theta]}$ Imag[2, 3, 4, 1]]

Out[18]=

4

In[19]:= PermMax[GenSet[{Cycles[{{1, 3}, {2, 5}}], $e^{i \operatorname{Re}[\theta]}$ }]

Out[19]=

5

ToCycl (PermutationCycles) and ToImag (PermutationList)

In[20]:= PermutationCycles[{4, 3, 1, 2, 7, 6, 5}]

Out[20]=

Cycles[{{1, 4, 2, 3}, {5, 7}}]

In[21]:= **ToCycl::usage**

Out[21]=

ToCycl[iper] converts an Imag permutation 'iper' or a standard permutation list into its CyclesPhased form {Cycles[...], sign}.

In[22]:= **Imag::usage**

Out[22]=

Imag[n1, n2, ...] is a head representing a permutation by its image list (the result of PermutationList).

In[23]:= ToCycl[$e^{i \operatorname{Re}[\theta]}$ Imag[4, 3, 1, 2, 7, 6, 5]] (* Imag → CyclesPhased *)

Out[23]=

{Cycles[{{1, 4, 2, 3}, {5, 7}}], $e^{i \operatorname{Re}[\theta]}$ }

In[24]:= ToCycl[{4, 3, 1, 2, 7, 6, 5}] (* imag_List → CyclesPhased *)

Out[24]=

{Cycles[{{1, 4, 2, 3}, {5, 7}}], 1}

Imag 표기는 순열의 크기에 대한 정보를 포함하지만 Cycles 표기는 크기 정보를 포함하지 않으므로, 함수에

Cycles를 전달할 때는 순열의 크기를 함수의 인자로 전달해야 한다. 단, 크기 인자가 없으면 PermMax를 이용하여 구한 값을 순열의 크기로 사용한다.

```
In[25]:= PermutationList[Cycles[{{1, 2, 3}, {4, 7}}]]
Out[25]= {2, 3, 1, 7, 5, 6, 4}

In[26]:= % // Length
Out[26]= 7

In[27]:= ToImag[{Cycles[{{1, 2, 3}, {4, 7}}], ei Re[θ]}] (* CyclesPhased → Imag *)
Out[27]= ei Re[θ] Imag[2, 3, 1, 7, 5, 6, 4]

In[28]:= ToImag[{Cycles[{{1, 2, 3}, {4, 7}}], ei Re[θ]}, 10]
Out[28]= ei Re[θ] Imag[2, 3, 1, 7, 5, 6, 4, 8, 9, 10]

In[29]:= ToImag[{{1, 2, 3}, {4, 7}}, 10] (* cycl_List → Imag *)
Out[29]= Imag[2, 3, 1, 7, 5, 6, 4, 8, 9, 10]
```

Rutines for STensor

mPerm 패키지는 STensor 패키지에서 주로 사용한다.

toCyclesSigned, toCyclesProper, gensetToPermGroup (PermutationGroup)

A 'CyclesSigned' is defined by a list {Cycles[{{...}}, +1|-1]}.

```
In[30]:= (* Cycles → CyclesSigned *)
PP`toCyclesSigned[Cycles[{{2, 4, 3}, {5, 6}}], 4] (* s = -1 *)
Out[30]= {Cycles[{{2, 4, 3}}], -1}

In[31]:= PP`toCyclesSigned[Cycles[{{2, 4, 3}}], 4] (* s = +1 *)
Out[31]= {Cycles[{{2, 4, 3}}], 1}
```

A 'CyclesProper' is a proper Cycles in which the last element plays the role the sign, + or -.

```
In[32]:= (* CyclesSigned → CyclesProper *)
PP`toCyclesProper[{Cycles[{{1, 4, 2, 3}}], -1}, 4]
```

```
Out[32]= Cycles[{{1, 4, 2, 3}, {5, 6}}]
```

```
In[33]:= (* CyclesProper → CyclesSigned *)
PP`toCyclesSigned[Cycles[{{1, 4, 2, 3}, {5, 6}}], 4]
```

```
Out[33]= {Cycles[{{1, 4, 2, 3}}], -1}
```

```
In[34]:= (* 'toCyclesProper' is for a 'CyclesSigned', not for a 'CyclesPhased'. *)
PP`toCyclesProper[{Cycles[{{1, 4, 2, 3}}],  $e^{i \operatorname{Re}[\theta]}$ }, 4]
```

```
Out[34]= mGRG`mPerm`Private`toCyclesProper[{Cycles[{{1, 4, 2, 3}}],  $e^{i \operatorname{Re}[\theta]}$ }, 4]
```

```
In[35]:= (* GenSet of CyclesSigned → PermutationGroup *)
PP`gensetToPermGroup[ToCycl /@ GenSet[-Imag[1, 4, 2, 3]], 4]
```

```
Out[35]= PermutationGroup[{Cycles[{{2, 4, 3}, {5, 6}}]]}
```

InversePerm (InversePermutation)

```
In[36]:= InversePermutation[Cycles[{{2, 3}, {5, 6, 1}}]]
```

```
Out[36]= Cycles[{{1, 6, 5}, {2, 3}}]
```

```
In[37]:= InversePerm::usage
```

```
Out[37]= InversePerm[perm] returns the inverse of the specified permutation.
```

```
In[38]:= InversePerm[{Cycles[{{2, 3}, {5, 6, 1}}],  $e^{i \operatorname{Re}[\theta]}$ }]
```

```
Out[38]= {Cycles[{{1, 6, 5}, {2, 3}}],  $e^{-i \operatorname{Re}[\theta]}$ }
```

```
In[39]:= InversePerm[{Cycles[{}],  $e^{i \operatorname{Re}[\theta]}$ }]
```

```
Out[39]= {Cycles[{}],  $e^{-i \operatorname{Re}[\theta]}$ }
```

```
In[40]:= InversePerm[-Imag[2, 3, 1]]
```

```
Out[40]= -Imag[3, 1, 2]
```

MakePermGroup (GroupElements)

In[41]:= **MakePermGroup::usage**

Out[41]=

MakePermGroup[gs, len] constructs a **PermutationGroup** object from a **GenSet** 'gs' acting on 'len' elements. It returns all elements of the group.

Returns all elements of the permutation group generated from **GenSet** of **CyclesSigned**:

In[42]:= **MakePermGroup**[**GenSet**[{**Cycles**[{{1, 2, 3, 4}}], 1}]]

Out[42]=

```
{{Cycles[{}], 1}, {Cycles[{{1, 2, 3, 4}}], 1},
 {Cycles[{{1, 3}, {2, 4}}], 1}, {Cycles[{{1, 4, 3, 2}}], 1}}
```

In[43]:= **MakePermGroup**[**GenSet**[{**Cycles**[{{1, 2}}], -1}, {**Cycles**[{{2, 3}}], 1}]]

Out[43]=

```
{{Cycles[{}], 1}, {Cycles[{}], -1}, {Cycles[{{2, 3}}], 1},
 {Cycles[{{2, 3}}], -1}, {Cycles[{{1, 2}}], 1}, {Cycles[{{1, 2}}], -1},
 {Cycles[{{1, 2, 3}}], 1}, {Cycles[{{1, 2, 3}}], -1}, {Cycles[{{1, 3, 2}}], 1},
 {Cycles[{{1, 3, 2}}], -1}, {Cycles[{{1, 3}}], 1}, {Cycles[{{1, 3}}], -1}}
```

In[44]:= **ToImag**[#, 3] & /@

MakePermGroup[**GenSet**[{**Cycles**[{{1, 2}}], -1}, {**Cycles**[{{2, 3}}], 1}]]

Out[44]=

```
{Imag[1, 2, 3], -Imag[1, 2, 3], Imag[1, 3, 2], -Imag[1, 3, 2],
 Imag[2, 1, 3], -Imag[2, 1, 3], Imag[2, 3, 1], -Imag[2, 3, 1],
 Imag[3, 1, 2], -Imag[3, 1, 2], Imag[3, 2, 1], -Imag[3, 2, 1]}
```

In[45]:= **List @@ Plus @@ %**

Out[45]=

0

In[46]:= **MakePermGroup**[**GenSet**[{**Cycles**[{{1, 2}}], -1}, {**Cycles**[{{2, 3}}], 1}]] ==

MakePermGroup[**GenSet**[{**Cycles**[{{2, 3}}], 1}, {**Cycles**[{{1, 2}}], -1}]]

Out[46]=

True

In[47]:= **MakePermGroup**[GenSet[{Cycles[{{1, 4, 3, 2}}], 1}, {Cycles[{{1, 2}}], 1}]]

Out[47]=

```
{Cycles[{}], 1}, {Cycles[{{3, 4}}], 1}, {Cycles[{{2, 3}}], 1},
{Cycles[{{2, 3, 4}}], 1}, {Cycles[{{2, 4, 3}}], 1}, {Cycles[{{2, 4}}], 1},
{Cycles[{{1, 2}}], 1}, {Cycles[{{1, 2}, {3, 4}}], 1},
{Cycles[{{1, 2, 3}}], 1}, {Cycles[{{1, 2, 3, 4}}], 1},
{Cycles[{{1, 2, 4, 3}}], 1}, {Cycles[{{1, 2, 4}}], 1},
{Cycles[{{1, 3, 2}}], 1}, {Cycles[{{1, 3, 4, 2}}], 1}, {Cycles[{{1, 3}}], 1},
{Cycles[{{1, 3, 4}}], 1}, {Cycles[{{1, 3}, {2, 4}}], 1},
{Cycles[{{1, 3, 2, 4}}], 1}, {Cycles[{{1, 4, 3, 2}}], 1},
{Cycles[{{1, 4, 2}}], 1}, {Cycles[{{1, 4, 3}}], 1}, {Cycles[{{1, 4}}], 1},
{Cycles[{{1, 4, 2, 3}}], 1}, {Cycles[{{1, 4}, {2, 3}}], 1}
```

In[48]:= **ToImag**[#, 4] & /@

MakePermGroup[GenSet[{Cycles[{{1, 4, 3, 2}}], 1}, {Cycles[{{1, 2}}], 1}]]

Out[48]=

```
{Imag[1, 2, 3, 4], Imag[1, 2, 4, 3], Imag[1, 3, 2, 4], Imag[1, 3, 4, 2],
Imag[1, 4, 2, 3], Imag[1, 4, 3, 2], Imag[2, 1, 3, 4], Imag[2, 1, 4, 3],
Imag[2, 3, 1, 4], Imag[2, 3, 4, 1], Imag[2, 4, 1, 3], Imag[2, 4, 3, 1],
Imag[3, 1, 2, 4], Imag[3, 1, 4, 2], Imag[3, 2, 1, 4], Imag[3, 2, 4, 1],
Imag[3, 4, 1, 2], Imag[3, 4, 2, 1], Imag[4, 1, 2, 3], Imag[4, 1, 3, 2],
Imag[4, 2, 1, 3], Imag[4, 2, 3, 1], Imag[4, 3, 1, 2], Imag[4, 3, 2, 1]}
```

In[49]:= **Sort**[**ToImag**[#, 4] & /@ **MakePermGroup**[

GenSet[{Cycles[{{1, 4, 3, 2}}], 1}, {Cycles[{{1, 2}}], 1}]]] ==

Sort[**Apply**[**Imag**, #] & /@ **Permutations**[{1, 2, 3, 4}]]

Out[49]=

True

In[50]:= **MakePermGroup**[GenSet[{Cycles[{{1, 5, 4, 3, 2}}], 1}, {Cycles[{{1, 2}}], 1}]]

Out[50]=

```
{Cycles[{}], 1}, {Cycles[{{4, 5}}], 1}, {Cycles[{{3, 4}}], 1},
{Cycles[{{3, 4, 5}}], 1}, {Cycles[{{3, 5, 4}}], 1}, {Cycles[{{3, 5}}], 1},
{Cycles[{{2, 3}}], 1}, {Cycles[{{2, 3}, {4, 5}}], 1},
{Cycles[{{2, 3, 4}}], 1}, {Cycles[{{2, 3, 4, 5}}], 1},
{Cycles[{{2, 3, 5, 4}}], 1}, {Cycles[{{2, 3, 5}}], 1},
{Cycles[{{2, 4, 3}}], 1}, {Cycles[{{2, 4, 5, 3}}], 1}, {Cycles[{{2, 4}}], 1},
{Cycles[{{2, 4, 5}}], 1}, {Cycles[{{2, 4}, {3, 5}}], 1},
{Cycles[{{2, 4, 3, 5}}], 1}, {Cycles[{{2, 5, 4, 3}}], 1},
{Cycles[{{2, 5, 3}}], 1}, {Cycles[{{2, 5, 4}}], 1}, {Cycles[{{2, 5}}], 1},
{Cycles[{{2, 5, 3, 4}}], 1}, {Cycles[{{2, 5}, {3, 4}}], 1},
{Cycles[{{1, 2}}], 1}, {Cycles[{{1, 2}, {4, 5}}], 1},
{Cycles[{{1, 2}, {3, 4}}], 1}, {Cycles[{{1, 2}, {3, 4, 5}}], 1},
{Cycles[{{1, 2}, {3, 5, 4}}], 1}, {Cycles[{{1, 2}, {3, 5}}], 1},
{Cycles[{{1, 2, 3}}], 1}, {Cycles[{{1, 2, 3}, {4, 5}}], 1},
```

```

{Cycles[{{1, 2, 3, 4}}, 1], {Cycles[{{1, 2, 3, 4, 5}}, 1],
{Cycles[{{1, 2, 3, 5, 4}}, 1], {Cycles[{{1, 2, 3, 5}}, 1],
{Cycles[{{1, 2, 4, 3}}, 1], {Cycles[{{1, 2, 4, 5, 3}}, 1],
{Cycles[{{1, 2, 4}}, 1], {Cycles[{{1, 2, 4, 5}}, 1],
{Cycles[{{1, 2, 4}, {3, 5}}, 1], {Cycles[{{1, 2, 4, 3, 5}}, 1],
{Cycles[{{1, 2, 5, 4, 3}}, 1], {Cycles[{{1, 2, 5, 3}}, 1],
{Cycles[{{1, 2, 5, 4}}, 1], {Cycles[{{1, 2, 5}}, 1],
{Cycles[{{1, 2, 5, 3, 4}}, 1], {Cycles[{{1, 2, 5}, {3, 4}}, 1],
{Cycles[{{1, 3, 2}}, 1], {Cycles[{{1, 3, 2}, {4, 5}}, 1],
{Cycles[{{1, 3, 4, 2}}, 1], {Cycles[{{1, 3, 4, 5, 2}}, 1],
{Cycles[{{1, 3, 5, 4, 2}}, 1], {Cycles[{{1, 3, 5, 2}}, 1],
{Cycles[{{1, 3}}, 1], {Cycles[{{1, 3}, {4, 5}}, 1],
{Cycles[{{1, 3, 4}}, 1], {Cycles[{{1, 3, 4, 5}}, 1],
{Cycles[{{1, 3, 5, 4}}, 1], {Cycles[{{1, 3, 5}}, 1],
{Cycles[{{1, 3}, {2, 4}}, 1], {Cycles[{{1, 3}, {2, 4, 5}}, 1],
{Cycles[{{1, 3, 2, 4}}, 1], {Cycles[{{1, 3, 2, 4, 5}}, 1],
{Cycles[{{1, 3, 5, 2, 4}}, 1], {Cycles[{{1, 3, 5}, {2, 4}}, 1],
{Cycles[{{1, 3}, {2, 5, 4}}, 1], {Cycles[{{1, 3}, {2, 5}}, 1],
{Cycles[{{1, 3, 2, 5, 4}}, 1], {Cycles[{{1, 3, 2, 5}}, 1],
{Cycles[{{1, 3, 4}, {2, 5}}, 1], {Cycles[{{1, 3, 4, 2, 5}}, 1],
{Cycles[{{1, 4, 3, 2}}, 1], {Cycles[{{1, 4, 5, 3, 2}}, 1],
{Cycles[{{1, 4, 2}}, 1], {Cycles[{{1, 4, 5, 2}}, 1],
{Cycles[{{1, 4, 2}, {3, 5}}, 1], {Cycles[{{1, 4, 3, 5, 2}}, 1],
{Cycles[{{1, 4, 3}}, 1], {Cycles[{{1, 4, 5, 3}}, 1], {Cycles[{{1, 4}}, 1],
{Cycles[{{1, 4, 5}}, 1], {Cycles[{{1, 4}, {3, 5}}, 1],
{Cycles[{{1, 4, 3, 5}}, 1], {Cycles[{{1, 4, 2, 3}}, 1],
{Cycles[{{1, 4, 5, 2, 3}}, 1], {Cycles[{{1, 4}, {2, 3}}, 1],
{Cycles[{{1, 4, 5}, {2, 3}}, 1], {Cycles[{{1, 4}, {2, 3, 5}}, 1],
{Cycles[{{1, 4, 2, 3, 5}}, 1], {Cycles[{{1, 4, 2, 5, 3}}, 1],
{Cycles[{{1, 4, 3}, {2, 5}}, 1], {Cycles[{{1, 4}, {2, 5, 3}}, 1],
{Cycles[{{1, 4, 3, 2, 5}}, 1], {Cycles[{{1, 4}, {2, 5}}, 1],
{Cycles[{{1, 4, 2, 5}}, 1], {Cycles[{{1, 5, 4, 3, 2}}, 1],
{Cycles[{{1, 5, 3, 2}}, 1], {Cycles[{{1, 5, 4, 2}}, 1],
{Cycles[{{1, 5, 2}}, 1], {Cycles[{{1, 5, 3, 4, 2}}, 1],
{Cycles[{{1, 5, 2}, {3, 4}}, 1], {Cycles[{{1, 5, 4, 3}}, 1],
{Cycles[{{1, 5, 3}}, 1], {Cycles[{{1, 5, 4}}, 1], {Cycles[{{1, 5}}, 1],
{Cycles[{{1, 5, 3, 4}}, 1], {Cycles[{{1, 5}, {3, 4}}, 1],
{Cycles[{{1, 5, 4, 2, 3}}, 1], {Cycles[{{1, 5, 2, 3}}, 1],
{Cycles[{{1, 5, 4}, {2, 3}}, 1], {Cycles[{{1, 5}, {2, 3}}, 1],
{Cycles[{{1, 5, 2, 3, 4}}, 1], {Cycles[{{1, 5}, {2, 3, 4}}, 1],
{Cycles[{{1, 5, 3}, {2, 4}}, 1], {Cycles[{{1, 5, 2, 4, 3}}, 1],
{Cycles[{{1, 5, 3, 2, 4}}, 1], {Cycles[{{1, 5}, {2, 4, 3}}, 1],
{Cycles[{{1, 5, 2, 4}}, 1], {Cycles[{{1, 5}, {2, 4}}, 1]}

```

In[51]:=

ToImag[#, 5] & /@

MakePermGroup[GenSet[{Cycles[{{1, 5, 4, 3, 2}}, 1], {Cycles[{{1, 2}}, 1]]]

Out[51]=

```
{Imag[1, 2, 3, 4, 5], Imag[1, 2, 3, 5, 4], Imag[1, 2, 4, 3, 5],
Imag[1, 2, 4, 5, 3], Imag[1, 2, 5, 3, 4], Imag[1, 2, 5, 4, 3],
Imag[1, 3, 2, 4, 5], Imag[1, 3, 2, 5, 4], Imag[1, 3, 4, 2, 5],
Imag[1, 3, 4, 5, 2], Imag[1, 3, 5, 2, 4], Imag[1, 3, 5, 4, 2],
Imag[1, 4, 2, 3, 5], Imag[1, 4, 2, 5, 3], Imag[1, 4, 3, 2, 5],
Imag[1, 4, 3, 5, 2], Imag[1, 4, 5, 2, 3], Imag[1, 4, 5, 3, 2],
Imag[1, 5, 2, 3, 4], Imag[1, 5, 2, 4, 3], Imag[1, 5, 3, 2, 4],
Imag[1, 5, 3, 4, 2], Imag[1, 5, 4, 2, 3], Imag[1, 5, 4, 3, 2],
Imag[2, 1, 3, 4, 5], Imag[2, 1, 3, 5, 4], Imag[2, 1, 4, 3, 5],
Imag[2, 1, 4, 5, 3], Imag[2, 1, 5, 3, 4], Imag[2, 1, 5, 4, 3],
Imag[2, 3, 1, 4, 5], Imag[2, 3, 1, 5, 4], Imag[2, 3, 4, 1, 5],
Imag[2, 3, 4, 5, 1], Imag[2, 3, 5, 1, 4], Imag[2, 3, 5, 4, 1],
Imag[2, 4, 1, 3, 5], Imag[2, 4, 1, 5, 3], Imag[2, 4, 3, 1, 5],
Imag[2, 4, 3, 5, 1], Imag[2, 4, 5, 1, 3], Imag[2, 4, 5, 3, 1],
Imag[2, 5, 1, 3, 4], Imag[2, 5, 1, 4, 3], Imag[2, 5, 3, 1, 4],
Imag[2, 5, 3, 4, 1], Imag[2, 5, 4, 1, 3], Imag[2, 5, 4, 3, 1],
Imag[3, 1, 2, 4, 5], Imag[3, 1, 2, 5, 4], Imag[3, 1, 4, 2, 5],
Imag[3, 1, 4, 5, 2], Imag[3, 1, 5, 2, 4], Imag[3, 1, 5, 4, 2],
Imag[3, 2, 1, 4, 5], Imag[3, 2, 1, 5, 4], Imag[3, 2, 4, 1, 5],
Imag[3, 2, 4, 5, 1], Imag[3, 2, 5, 1, 4], Imag[3, 2, 5, 4, 1],
Imag[3, 4, 1, 2, 5], Imag[3, 4, 1, 5, 2], Imag[3, 4, 2, 1, 5],
Imag[3, 4, 2, 5, 1], Imag[3, 4, 5, 1, 2], Imag[3, 4, 5, 2, 1],
Imag[3, 5, 1, 2, 4], Imag[3, 5, 1, 4, 2], Imag[3, 5, 2, 1, 4],
Imag[3, 5, 2, 4, 1], Imag[3, 5, 4, 1, 2], Imag[3, 5, 4, 2, 1],
Imag[4, 1, 2, 3, 5], Imag[4, 1, 2, 5, 3], Imag[4, 1, 3, 2, 5],
Imag[4, 1, 3, 5, 2], Imag[4, 1, 5, 2, 3], Imag[4, 1, 5, 3, 2],
Imag[4, 2, 1, 3, 5], Imag[4, 2, 1, 5, 3], Imag[4, 2, 3, 1, 5],
Imag[4, 2, 3, 5, 1], Imag[4, 2, 5, 1, 3], Imag[4, 2, 5, 3, 1],
Imag[4, 3, 1, 2, 5], Imag[4, 3, 1, 5, 2], Imag[4, 3, 2, 1, 5],
Imag[4, 3, 2, 5, 1], Imag[4, 3, 5, 1, 2], Imag[4, 3, 5, 2, 1],
Imag[4, 5, 1, 2, 3], Imag[4, 5, 1, 3, 2], Imag[4, 5, 2, 1, 3],
Imag[4, 5, 2, 3, 1], Imag[4, 5, 3, 1, 2], Imag[4, 5, 3, 2, 1],
Imag[5, 1, 2, 3, 4], Imag[5, 1, 2, 4, 3], Imag[5, 1, 3, 2, 4],
Imag[5, 1, 3, 4, 2], Imag[5, 1, 4, 2, 3], Imag[5, 1, 4, 3, 2],
Imag[5, 2, 1, 3, 4], Imag[5, 2, 1, 4, 3], Imag[5, 2, 3, 1, 4],
Imag[5, 2, 3, 4, 1], Imag[5, 2, 4, 1, 3], Imag[5, 2, 4, 3, 1],
Imag[5, 3, 1, 2, 4], Imag[5, 3, 1, 4, 2], Imag[5, 3, 2, 1, 4],
Imag[5, 3, 2, 4, 1], Imag[5, 3, 4, 1, 2], Imag[5, 3, 4, 2, 1],
Imag[5, 4, 1, 2, 3], Imag[5, 4, 1, 3, 2], Imag[5, 4, 2, 1, 3],
Imag[5, 4, 2, 3, 1], Imag[5, 4, 3, 1, 2], Imag[5, 4, 3, 2, 1]}
```

In[52]:=

Length@**MakePermGroup[GenSet[{Cycles[{{1, 5, 4, 3, 2}}], 1}, {Cycles[{{1, 2}}], 1}]]**

Out[52]=

120

```
In[53]:= Length@MakePermGroup[
  GenSet[{Cycles[{{1, 6, 5, 4, 3, 2}}], 1}, {Cycles[{{1, 2}}], 1}]
Out[53]= 720
```

```
In[54]:= Timing[Length@MakePermGroup[
  GenSet[{Cycles[{{1, 7, 6, 5, 4, 3, 2}}], 1}, {Cycles[{{1, 2}}], 1}]]
Out[54]= {0.071521, 5040}
```

Orbits (GroupOrbits)

```
In[55]:= Orbits::usage
Out[55]= Orbits[pts, gs, len] finds the orbits of the points
  'pts' under the action of the group generated by GenSet 'gs'.
```

Returns the sorted orbits of the points under the action of the group generated by GenSet of CyclesSigned:

```
In[56]:= Orbits[{2, 5}, GenSet[{Cycles[{{1, 2}}], +1}, {Cycles[{{4, 5}}], +1}], 6]
Out[56]= {{1, 2}, {4, 5}}
```

```
In[57]:= Orbits[{2}, GenSet[{Cycles[{{1, 2}}], +1}, {Cycles[{{4, 5}}], +1}], 3]
Out[57]= {{1, 2}}
```

```
In[58]:= gs = GenSet[{Cycles[{{1, 3, 6}, {2, 4}}], +1}, {Cycles[{{6, 7}}], +1}];
Orbits[{3}, gs, 7]
Orbits[{7, 3, 4}, gs, 7]
Out[59]= {{1, 3, 6, 7}}
Out[60]= {{1, 3, 6, 7}, {2, 4}}
```

PermMemberQ (GroupElementQ)

```
In[61]:= PermMemberQ::usage
Out[61]= PermMemberQ[perm, len, gs] checks if a permutation
  'perm' is a member of the group generated by GenSet 'gs'.
```

In[62]:= PermMemberQ[{Cycles[{}], +1}, 1, GenSet[]]

Out[62]=

True

In[63]:= PermMemberQ[ToCycl@Imag[2, 1], 2, GenSet[]]

Out[63]=

False

In[64]:= gs = GenSet[{Cycles[{{1, 2}}], -1}, {Cycles[{{3, 4}}], -1},
 {Cycles[{{5, 6}}], -1}, {Cycles[{{7, 8}}], -1}, {Cycles[{{9, 10}}], -1},
 {Cycles[{{1, 3}, {2, 4}}], +1}, {Cycles[{{3, 5}, {4, 6}}], +1},
 {Cycles[{{5, 7}, {6, 8}}], +1}, {Cycles[{{7, 9}, {8, 10}}], +1}];

In[65]:= PermMemberQ[ToCycl[-Imag[2, 1, 4, 3, 6, 5, 7, 8, 9, 10]], 10, gs]

Out[65]=

True

In[66]:= PermMemberQ[{Cycles[{{5, 6}}], -1}, 7, gs]

... PermMemberQ: The specified length 7 is too small for the given permutations.

Out[66]=

False

In[67]:= PermMemberQ[{Cycles[{{5, 6}}], -1}, 10, gs]

Out[67]=

True

PermuteList (Permute)

In[68]:= PermuteList::usage

Out[68]=

PermuteList[list, perm] permutes
 the 'list' according to the permutation 'perm'.

In[69]:= Permute[{1, 3, 4, 2}, Cycles[{{1, 2}}]]

Out[69]=

{3, 1, 4, 2}

In[70]:= PermuteList[{1, 3, 4, 2}, {Cycles[{{1, 2}}], $e^{i \operatorname{Re}[\theta]}$ }]

Out[70]=

{3, 1, 4, 2}

```
In[71]:= PermuteList[{1, 3, 4, 2},  $e^{\frac{i}{2} \text{Re}[\theta]}$  Imag[2, 1]]
```

```
Out[71]=
```

```
{3, 1}
```

```
In[72]:= PermuteList[{1, 3, 4, 2}, {2, 1}]
```

```
Out[72]=
```

```
{3, 1}
```